Experiment No. 5
Explore the working of any pre-trained model towards outcome generation.
Date of Performance:
Date of Submission:
Marks:
Sign:

Aim: Explore the working of any pre-trained model towards outcome generation.

Objective:

• To understand the concept of pre-trained models.
• To explore transfer learning in deep learning.
• To use a pre-trained model for prediction tasks.
• To analyze the performance of the model on given data.

Theory:

Pre-trained Models:

Pre-trained models are deep learning models that are already trained on large datasets (such as ImageNet). These models can be reused for solving similar problems without training from scratch.

Transfer Learning:

Transfer learning is a technique where knowledge gained from one task is applied to another related task. It helps in:

- Reducing training time
- Improving performance
- Working with smaller datasets

Examples of Pre-trained Models:

- VGG16
- ResNet
- MobileNet
- Inception

Working:

1. Load a pre-trained model
2. Use it directly or fine-tune it
3. Provide input data
4. Generate predictions

Implementation:

• Import required libraries such as TensorFlow/Keras.

• Load a pre-trained model (e.g., MobileNet or VGG16).

• Preprocess input data according to model requirements.

• Load and prepare dataset (images).

• Use the model to perform prediction.

• Analyze output results.

Code:

```
# Install dependencies:

# pip install tensorflow pillow matplotlib numpy


import tensorflow as tf

import numpy as np

from tensorflow.keras.applications import MobileNetV2

from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

from tensorflow.keras.preprocessing import image

import matplotlib.pyplot as plt


# Step 1: Load pre-trained model
```

```
model = MobileNetV2(weights='imagenet')


# Step 2: Load and preprocess image

img_path = 'sample.jpg'  # replace with your image file

img = image.load_img(img_path, target_size=(224, 224))


plt.imshow(img)

plt.title("Input Image")

plt.axis('off')

plt.show()


x = image.img_to_array(img)

x = np.expand_dims(x, axis=0)

x = preprocess_input(x)


# Step 3: Make prediction

preds = model.predict(x)


# Step 4: Decode predictions

decoded = decode_predictions(preds, top=3)[0]


print("Top Predictions:")
```

```
for i, (imagenetID, label, prob) in enumerate(decoded):
    print(f"{i+1}: {label} ({prob*100:.2f}%)")
```

Conclusion:

Pre-trained models are efficient for solving real-world problems. They reduce computational cost and training time while providing high accuracy through transfer learning.